Spring Boot - Top 30 Concepts

1 Spring vs Spring Boot

- Spring Boot is built on top of Spring to make development
- faster, easier and production ready.
- It removes the need for complex configuration.

Feature	Spring	Spring Boot
Configuration	Manual (XML/Java)	Auto-configuration
Setup	Complex	Simpler
Dependency Management	Manual	Uses Starters
Embedded Server	Needs external server (Tomcat)	Embedded server (Tomcat, Jetty)
Production Ready	Needs extra setup	Built-in features (Actuator, etc.)

Key Takeaways:

- Spring Boot = convention over configuration.
- Faster development and easy to maintain.
- Widely used for microservices and cloud applications.

---

2 Spring Boot Project Structure

- A Spring Boot project follows a standard structure.
- Code is organized in layers for better maintainability.

```
springboot-app/
├── src/
│   └── main/
│       ├── java/
│       │   └── com/example/demo/
│       │       ├── controller/
│       │       ├── service/
│       │       ├── repository/
│       │       ├── model/
│       │       └── DemoApplication.java
│       └── resources/
│           ├── application.properties (or .yml)
│           ├── static/
│           └── templates/
└── test/
    pom.xml (Maven) or build.gradle (Gradle)
```

Important Files / Folders

Path	Purpose
DemoApplication.java	Main class (contains main method)
controller/	REST controllers (handle requests)
service/	Business logic
repository/	Database operations
model/	Entity / DTO classes
application.properties	Configuration file
static/	Static files (CSS, JS, images)
templates/	View templates (e.g. Thymeleaf)
pom.xml / build.gradle	Dependency management

Tip: Keep the main class in the root package so that component scanning works for all sub-packages.

3 Spring Boot Starters

- Starters are pre-configured dependency sets.
- They simplify dependency management.
- Each starter includes commonly used libraries.
- Naming convention: spring-boot-starter-<name>

```
<!-- pom.xml -->
<dependency>
   <groupId>org.springframmerk.boot</groupId>
   <artifactId>spring-boot-starter-web./artifactId>
</dependency>
```

Common Starters

Starter	Purpose
spring-boot-starter-web	Build REST APIs (Spring MVC, Tomcat, Jackson)
spring-boot-starter-data-jpa	JPA + Hibernate
spring-boot-starter-security	Security & Authentication
spring-boot-starter-test	Testing (JUnit, Mockito)
spring-boot-starter-actuator	Monitoring & Health checks
spring-boot-starter-validation	Input validation (Hibernate Validator)

Key Points:

- Reduce boilerplate dependency configuration.
- Managed by Spring Boot.
- Keep the project lightweight.

Use Cases:

- Quickly add web, data, security, test dependencies.
- Standard setup for most Spring Boot projects.
- Helps maintain compatible library versions.

---

4 Auto-Configuration

- Spring Boot automatically configures beans based on classpath.
- It uses sensible defaults and convention over configuration.
- Reduces manual XML/Java configuration.
- You can also customize or exclude auto-configuration if needed.

How it works?



Example (Disable auto-configuration)

```
@SpringBootApplication(
   exclude = { DataSourceAutoConfiguration.class }
)
public class DemoApplication {
   public static void main(String[] args) {
      SpringApplication.run(DemoApplication.class, args);
   }
}
```

Key Points:

- Works based on classpath and conditions.
- Uses @ConditionalOnClass, @ConditionalOnMissingBean etc. internally.
- You can exclude or customize auto-configuration.

Use Cases:

- Faster setup and development.
- Reduce manual configuration.
- Easily customize default behavior.

Spring Boot - Top 30 Concepts

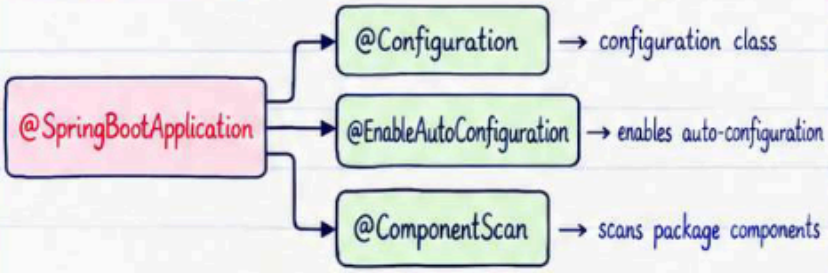
Key Ideas Behind Modern Spring Applications

5 @SpringBootApplication

Main annotation to start a Boot app

- It is the main annotation on the bootstrapping class.
- It combines three important annotations.

How it works?



Example (Bootstrapping class)

```
@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

Key Points

- Entry point of a Spring Boot application.
- Reduces setup by combining multiple annotations.
- Usually placed in the root package.

---

6 Dependency Injection (DI)

Provide dependencies to classes

- DI lets Spring provide the required objects to a class.
- It reduces tight coupling between components.

How it works?



Example (Constructor Injection)

```
@Service
public class UserService {
    private final UserRepository repo;

    public UserService(UserRepository repo) {
        this.repo = repo;
    }
}
```

Key Points

- Promotes loose coupling between components.
- Spring manages the dependencies.
- Constructor injection is preferred (immutability & testability).

Spring Boot - Top 30 Concepts

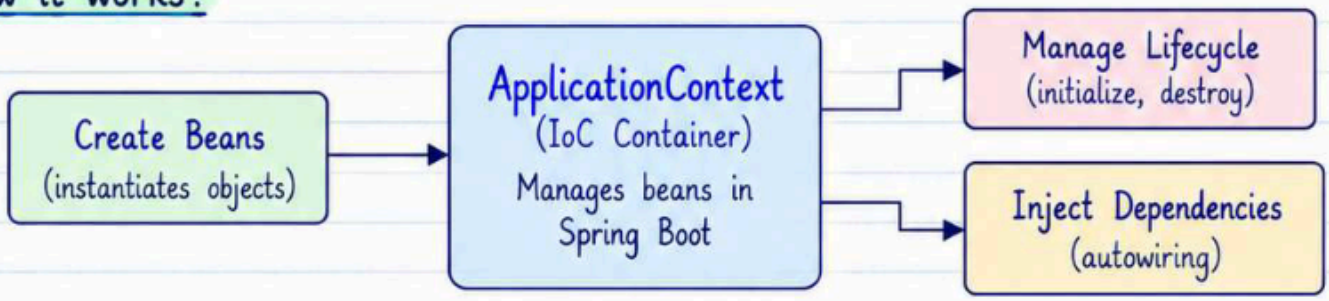
Key Ideas Behind Modern Spring Applications

7 IoC Container & ApplicationContext

Container that manages Spring beans

- Inversion of Control (IoC) means control is given to Spring to create and manage objects.
- ApplicationContext is the main container in Spring Boot (extends IoC container).

How it works?



Key Points

- Central container that manages application beans.
- Handles bean creation, lifecycle and dependency injection.
- Provides extra features like event handling, internationalization, etc.

---

8 Beans & @Bean

Objects managed by Spring

- Beans are objects managed by the Spring container.
- @Bean is used in @Configuration classes to define beans.

Example

```
@Configuration
public class AppConfig {
    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}
```

How it works?



Key Points

- Defined in a @Configuration class.
- Useful for creating third-party objects (e.g. RestTemplate).
- Lifecycle is managed by Spring.

Spring Boot - Top 30 Concepts

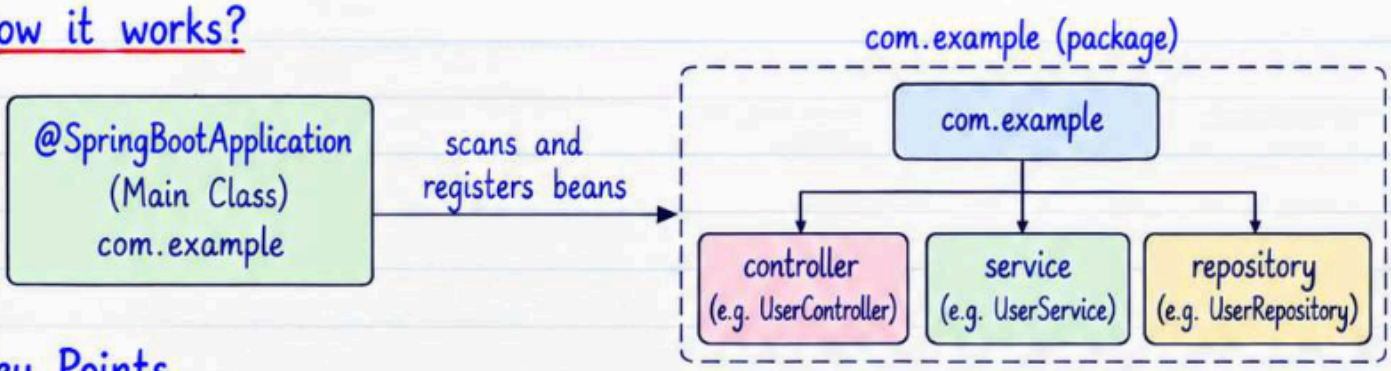
Key Ideas Behind Modern Spring Applications

9 Component Scanning

Automatically find Spring components

- Spring scans packages to find classes annotated with Spring stereotypes (e.g. @Component).
- By default, it scans the package of the main class and all its subpackages.

How it works?



Key Points

- Automatic detection of Spring components.
- Configurable base packages using @ComponentScan.
- Reduces manual bean registration.

---

10 Stereotype Annotations

Mark classes for specific roles

- Stereotype annotations mark classes so Spring can detect and register them as beans.
- They are specializations of @Component, each serving a specific purpose.

Annotation	Purpose
@Component	Generic Spring component (any layer)
@Service	Service layer / business logic
@Repository	Data access layer (e.g. DAO, database operations)
@Controller	MVC controller (returns views)
@RestController	Controller + @ResponseBody (returns JSON)

Key Points

- Specializations of @Component.
- Improve code readability and maintainability.
- Help with automatic scanning and bean registration.

Spring Boot - Top 30 Concepts

Key Ideas Behind Modern Spring Applications

11 Configuration Properties

Read app settings from config files

- Application settings are stored in `application.properties` or `application.yml`.
- These properties can be bound to Java classes for easy access.

Example Configuration (application.yml)

```yaml
app:
  name: Demo App
  version: 1.0
```

Access in Java

```java
@Value("${app.name}")
private String appName;
```

Key Points

- Externalized configuration (keeps settings outside code).
- Type-safe binding with `@ConfigurationProperties`.
- Keeps code flexible and easy to maintain.

---

12 Profiles & Environments

Different config for dev, test, prod

- Spring Boot allows different configuration files for different environments.
- You can activate a profile to load the respective settings.

application-dev.yml

```yaml
server:
  port: 8081
```

application-prod.yml

```yaml
server:
  port: 8080
```

Activate Profile

```properties
# In application.properties
spring.profiles.active=dev
```

Key Points

- Separate configuration for each environment.
- Avoid hardcoding environment-specific values.
- Common profiles: dev, test, prod.

Spring Boot - Top 30 Concepts

Key Ideas Behind Modern Spring Applications

13 REST Controllers

Handle HTTP requests and return data

- @RestController is used to create REST APIs in Spring Boot.
- It returns data as JSON by default (no need for @ResponseBody).

Example Code:

```
@RestController
@RequestMapping("/api/users")
public class UserController {
    @GetMapping
    public List<String> getUsers() {
        return List.of("Aman", "Rahul");
    }
}
```

How it works?



Key Points

- Used to build REST APIs.
- Maps HTTP endpoints to controller methods.
- Returns JSON responses by default.

14 Request Mapping & Parameters

Map URLs and read inputs

- Use mapping annotations to define URLs and handle input parameters.
- Supports path variables and query parameters to read data from requests.

Example Code:

```
@RestController
@RequestMapping("/api/users")
public class UserController {
    @GetMapping("/users/{id}")
    public String getUser(
        @PathVariable Long id,
        @RequestParam(required = false) String name) {
        return "User: " + id + ", name: " + name;
    }
}
```

Common Annotations

```
@GetMapping
@PostMapping
@PutMapping
@DeleteMapping
@PathVariable
@RequestParam
```

Key Points

- Maps requests to controller methods.
- Handles path variables and query parameters.
- Supports all HTTP methods (GET, POST, PUT, DELETE).

15 Request Body & Validation

Receive JSON and validate input

- @RequestBody converts JSON from the request into a Java object.
- Validation annotations help check input and ensure data is valid.

DTO (with validation)

```
public class CreateUserRequest {
    @NotBlank
    private String name;

    @Email
    private String email;
}
```

Controller Method

```
public ResponseEntity<String> create(
  @Valid @RequestBody CreateUserRequest req) {
  // create user
  return ResponseEntity.ok("User created");
}
```

Key Points

- Converts JSON request to a Java object.
- @Valid triggers validation annotations.
- Catches bad input early and prevents invalid data.

---

16 ResponseEntity & HTTP Status Codes

Control response body, headers and status

- ResponseEntity lets you control the response body, headers and HTTP status code.
- Use appropriate status codes to indicate the result of the request.

Code Example

```
return ResponseEntity.ok(user);
return ResponseEntity.status(201).body("Created");
```

Common Status Codes

Status Code	Reason Phrase	Use Case
200	OK	Successful request
201	Created	New resource created
400	Bad Request	Invalid input
404	Not Found	Resource not found
500	Internal Server Error	Server error

Key Points

- Full control over response body, headers and status code.
- Helps build better and more flexible APIs.
- Clear communication with clients using standard status codes.

Spring Boot - Top 30 Concepts

Key Ideas Behind Modern Spring Applications

17 Exception Handling

Handle API errors cleanly

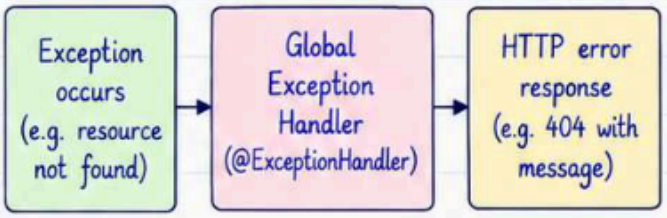
- @ControllerAdvice handles exceptions globally across the application.
- @ExceptionHandler handles specific exceptions and returns custom responses.

Example Code:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<String> handleNotFound(
        ResourceNotFoundException ex) {
        return ResponseEntity.status(404).body(ex.getMessage());
    }
}
```

How it works?



Key Points

- Centralized exception handling.
- Returns meaningful and consistent responses.
- Less try-catch code in controllers.

---

18 Filters vs Interceptors

Request pre/post processing

- Filters work at the servlet level, while interceptors work at the Spring MVC level.
- Both can be used for tasks like authentication, logging, and modifying requests/responses.

Comparison

Aspect	Filter	Interceptor
Level	Servlet level	Spring MVC level
Use case	Auth, logging, CORS	Controller-related tasks
Access	Raw request-response	Handler methods + model

Key Points

- Use Filter for low-level tasks (e.g. authentication, logging, CORS).
- Use Interceptor for application-level logic (e.g. modify model, check user roles).
- Both are useful and complement each other in the request flow.

19 Spring Data JPA

Simplify database access

- Spring Data JPA provides repository interfaces for database operations.
- It reduces boilerplate code for common CRUD operations.

Entity (example)

```
@Entity
public class User {
    @Id @GeneratedValue
    private Long id;
    private String name;
}
```

Repository (example)

```
public interface UserRepository
  extends JpaRepository<User, Long> {
}
```

Key Points

- Easy CRUD operations with minimal code.
- Supports query methods (e.g. findByName()).
- Built on top of JPA/Hibernate.

---

20 Entity Relationships

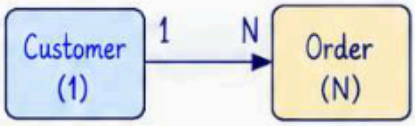
Map associations between tables

- Entity relationships define how entities are connected with each other.
- They help map real-world relationships between database tables.

Customer Entity (example)

```
@OneToMany(mappedBy = "customer")
private List<Order> orders;
```

Relationship (1 : N)



Order Entity (example)

```
@ManyToOne
@JoinColumn(name = "customer_id")
private Customer customer;
```

Key Points

- Choose the right mapping (OneToOne, OneToMany, ManyToOne, ManyToMany).
- Uses foreign keys to maintain relationships.
- Models real business relationships between entities.

21 Transactional (@Transactional)

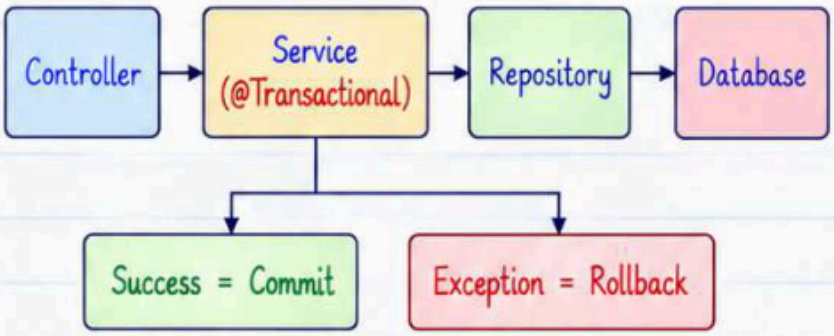
Ensure data consistency

- @Transactional wraps a method in a database transaction.
- It commits the changes if the method succeeds, and rolls back if a failure occurs.

Code Example

```
@Service
public class TransferService {
    @Transactional
    public void transfer() {
        debit();
        credit();
    }
}
```

Transaction Flow (example)



Key Points

- Applied mostly at the service layer.
- Maintains consistency across multiple DB operations.
- Rollback happens on runtime exceptions.

---

22 Pagination & Sorting

Fetch records in chunks

- Pagination returns data page by page from the database.
- Sorting orders the result based on one or more fields.

Code Example

```
Pageable pageable = PageRequest.of(
        0, 10, Sort.by("name").ascending());

Page<User> users = userRepository.findAll(pageable);
```

Page contains

Property	Description
content	list of records
totalElements	total rows
totalPages	page count

Key Points

- Improves performance for large datasets.
- Common in REST APIs.
- Supports page, size and sort parameters.

23 Spring Security Basics

Secure routes and users

- Spring Security protects endpoints using authentication and authorization.
- It works through a security filter chain.

Request Flow



Authentication vs Authorization

```
Authentication = who are you?
Authorization  = what can you access?
```

Security Config (example)

```
http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/public/**").permitAll()
    .anyRequest().authenticated());
```

Key Points

- Secures REST APIs and web apps.
- Supports roles and permissions.
- Integrates with login, JWT, or OAuth mechanisms.

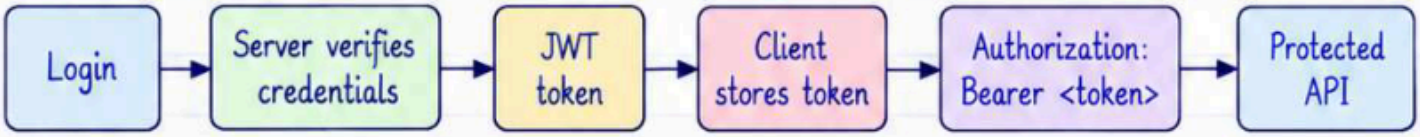
---

24 JWT Authentication

Stateless token-based auth

- User logs in once and receives a signed token.
- Client sends the token in later requests.

Authentication Flow



JWT Format

```
JWT = Header.Payload.Signature
```

Key Points

- Stateless authentication (no server-side session).
- Common for microservices and SPAs.
- Token must be validated on each request.

25 Testing in Spring Boot

Verify application behavior

- Spring Boot supports unit, slice, and integration testing.
- Different test annotations load different amounts of context.

Test Type	Annotation
Unit	Mockito/JUnit
Web layer	@WebMvcTest
Full app	@SpringBootTest

Test Class Example

```
@SpringBootTest
class UserServiceTest {
    @Autowired UserService service;
    @Test void testUsers() { }
}
```

Key Points

- You can test controllers, services, and repositories.
- Smaller tests (unit/slice) are faster and easier to maintain.
- Full integration tests are useful for end-to-end behavior.

---

26 Mocking with Mockito

Replace dependencies in tests

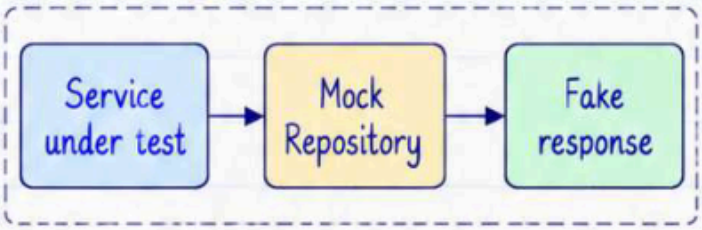
- Mocks simulate the behavior of real dependencies (e.g. repositories, external services).
- They help isolate business logic and make tests fast and reliable.

Code Example

```
@Mock UserRepository repo;
@InjectMocks UserService service;

when(repo.findById(1L)).thenReturn(Optional.of(user));
verify(repo).findById(1L);
```

How Mocking Works



Key Points

- Good for unit testing.
- Avoids real database or network calls.
- Use when/thenReturn to define behavior and verify to check interactions.

27 Actuator & Monitoring

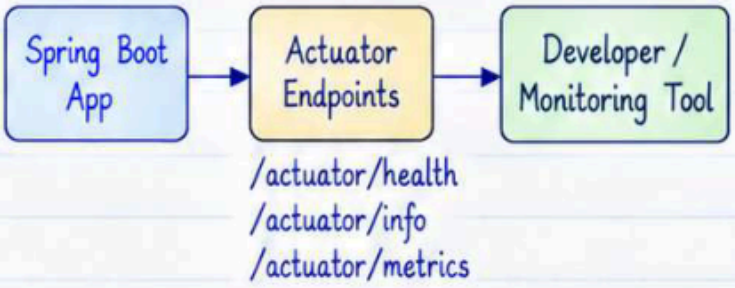
Observe app health and metrics

- Actuator exposes production-ready endpoints for health and monitoring.
- These endpoints help operations and debugging.

Common Endpoints

Endpoint	Purpose
/actuator/health	Check application health
/actuator/info	Show application info
/actuator/metrics	Expose application metrics

How It Works



Key Points

- Useful in production environments.
- Helps monitor application uptime and performance.
- Secure sensitive endpoints (e.g. with authentication).

---

28 Caching (@Cacheable)

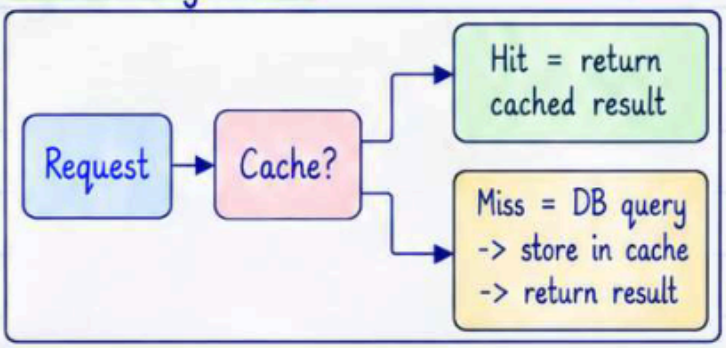
Store repeated results

- Caching stores frequently used results to avoid repeated database hits.
- Spring Boot supports annotations for cache abstraction.

Code Example

```
@EnableCaching
@Cacheable("users")
public User getUser(Long id) {
   return repo.findById(id).orElseThrow();
}
```

How Caching Works



Key Points

- Improves read performance.
- Reduces database load.
- Best for frequently accessed data.

29 Async & Scheduling

Run tasks later or in background

- @Async runs a method in a separate thread.
- @Scheduled runs a method automatically at fixed times or intervals.

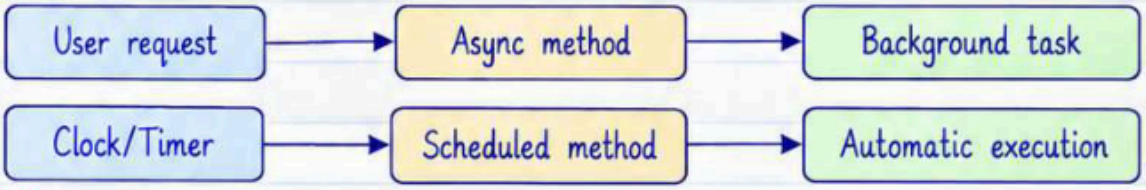
@Async (example)

```
@Async
public void sendEmail() {
   // send email in background
}
```

@Scheduled (example)

```
@Scheduled(cron = "0 0 9 * * *")
public void dailyReport() {
   // generate daily report
}
```

How It Works



Key Points

- Useful for sending emails, notifications, etc.
- Helps run background jobs without blocking the main thread.
- Ideal for periodic tasks like cleanup or report generation.

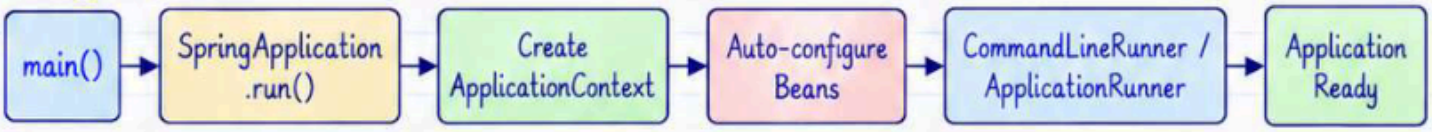
---

30 Spring Boot Application Lifecycle

From startup to ready state

- Spring Boot starts by creating an ApplicationContext and initializing beans.
- Hooks like runners can execute custom code after startup.

Lifecycle Flow



Lifecycle hooks (example)

```
@PostConstruct   // after bean creation
CommandLineRunner   // after application startup
```

Key Points

- Understand startup flow to debug issues easily.
- Runners (CommandLineRunner/ApplicationRunner) execute after context startup.
- Lifecycle knowledge helps in performing initialization tasks.